

PrintIt — Complete Project Documentation

Secure, Zero-Retention Cloud Printing Application

Table of Contents

- 1. Problem Statement
- 2. Solution Overview
- 3. System Architecture
- 4. Technology Stack (with Justifications)
- 5. Project Structure
- 6. Backend Deep Dive (server/server.js)
- 7. Shop Portal Deep Dive (shop-client/)
- 8. User App Deep Dive (user-client/)
- 9. The Six Security Mandates
- 10. End-to-End Data Flow
- 11. Redis Data Schemas
- 12. API Contracts & Socket Events
- 13. CSS Strategy: The Blur & Print System
- 14. Design System: The Vault-Atelier
- 15. Deployment Architecture
- 16. Key Bug Fixes & Production Patches
- 17. Security Edge Cases Handled
- 18. FAQ / Interview-Ready Explanations

1. Problem Statement

The Real-World Problem

When people visit local cyber cafes or print shops to print sensitive documents (Aadhar cards, PAN cards, legal papers, medical reports), they face a critical privacy risk:

- They hand over a **USB drive** or **email login** to the shop owner.
- The shop owner opens the file on their computer.
- After printing, the file **remains on the shop's computer** — in the Downloads folder, browser cache, recent files, temp directories, or even the Recycle Bin.
- The shop owner (or anyone with access to that computer) can **view, copy, or misuse** those sensitive documents at any time.

Why Existing Solutions Fail

Approach	Problem
USB Drive	File copied to local disk; remains after printing

Approach	Problem
Email (Gmail/WhatsApp)	Login credentials exposed; file downloaded to browser cache
Google Drive Link	File still opens in browser; can be downloaded/cached
AirDrop / Bluetooth	Requires same ecosystem; file still persists on receiver's device

The Core Challenge

How do you print a document on a stranger's computer without leaving ANY digital trace of that document on their machine?

2. Solution Overview

What is PrintIt?

PrintIt is a **privacy-first, zero-retention cloud printing web application** that allows users to print sensitive documents at any print shop without ever leaving a file on the shop's computer.

How It Works (Simple Explanation)

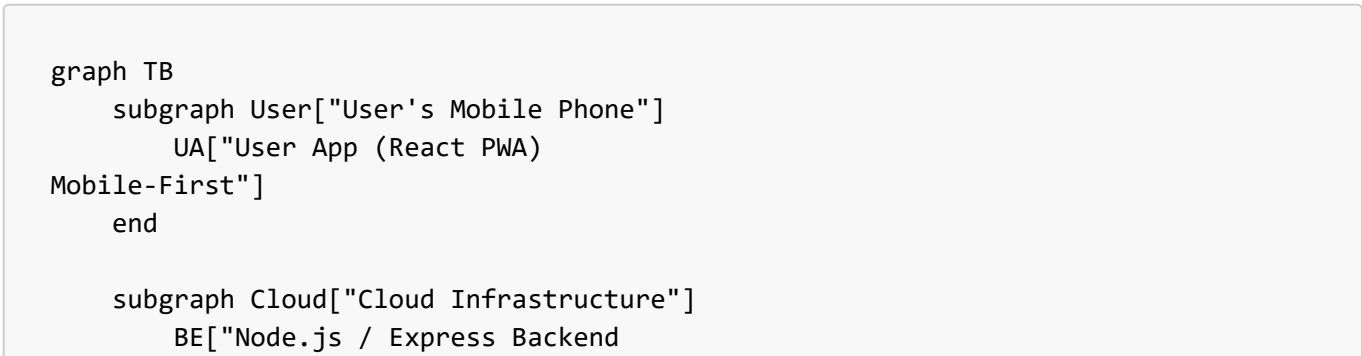
1. The **shop owner** opens a web page on their computer. A **6-digit code** appears on screen.
2. The **customer** opens the PrintIt app on their phone, uploads their document, and enters the 6-digit code.
3. The document is **instantly sent** to the shop's computer via an encrypted WebSocket tunnel.
4. The shop's browser **automatically opens the print dialog** — no manual action needed.
5. The moment printing finishes (or the dialog is closed), the document is **instantly destroyed** from everywhere: the browser DOM, the server's memory, and Redis.

Key Innovation: Zero Retention

- **No file is ever written to any disk** — not the server, not the shop's computer, not anywhere.
- Documents exist **only in RAM** (Redis + browser memory) for a maximum of 5 minutes.
- The document preview on the shop's screen is **blurred** so the owner can't photograph it.
- The blur is **removed only when sent to the physical printer** via a CSS `@media print` query.

3. System Architecture

High-Level Architecture Diagram



```
(Render.com)"]
    RD["Redis (Upstash)
    Ephemeral Memory Only"]
    end

    subgraph Shop["Shop's Desktop Computer"]
        SP["Shop Portal (React)
        Desktop-First"]
    end

    UA -->|"REST API
    POST /api/upload
    (multipart/form-data)"| BE
    BE <-->|"Read/Write
    TTL-enforced keys"| RD
    BE <-->|"WebSocket (Socket.io)
    Real-time events"| SP

    style UA fill:#005338,color:#fff
    style BE fill:#00488d,color:#fff
    style RD fill:#c43333,color:#fff
    style SP fill:#00488d,color:#fff
```

Communication Model: Hybrid REST + WebSocket

Channel	Used For	Why?
REST API (POST /api/upload)	Uploading the document file from the User App	multer requires standard HTTP multipart/form-data to parse file buffers in memory. WebSockets are not ideal for large binary uploads.
WebSocket (Socket.io)	Code generation, real-time document push, kill switch	Instantaneous bidirectional communication. The shop doesn't "poll" for documents — they are pushed the moment they're ready.

4. Technology Stack (with Justifications)

Backend Technologies

Technology	Version	Purpose	Why This Specific Choice?
Node.js	v18+	Server runtime	Non-blocking I/O is critical for handling simultaneous WebSocket connections from multiple shops. JavaScript on both frontend and backend simplifies the codebase.
Express.js	^4.x	HTTP framework	Lightweight, minimal, and perfectly suited for a single-route API (POST /api/upload). No need for heavyweight frameworks like NestJS.

Technology	Version	Purpose	Why This Specific Choice?
Socket.io	^4.x	Real-time WebSocket communication	Provides automatic reconnection, room-based targeting (crucial for <code>io.to(socketId)</code>), and fallback to HTTP long-polling if WebSocket isn't available. Raw <code>ws</code> would require building all of this manually.
ioredis	^5.x	Redis client for Node.js	Fastest Redis client for Node. Supports pipelining (batching multiple Redis commands), which we use to atomically write pairing codes and delete them. Chosen over the <code>redis</code> npm package for superior performance.
multer	^1.x	HTTP file upload parsing	The only Node.js middleware that supports <code>memoryStorage()</code> — keeping uploaded files as RAM Buffers without ever touching the disk. This is the foundation of our zero-retention promise.
cors	^2.x	Cross-Origin Resource Sharing	The frontend (Vercel) and backend (Render) are on different domains, so CORS headers must be explicitly set.
crypto	Built-in	Cryptographic operations	Node's native module. Used for <code>crypto.randomInt()</code> (6-digit codes) and <code>crypto.randomBytes()</code> (32-byte print tokens). We never use <code>Math.random()</code> because it is not cryptographically secure.

Frontend Technologies

Technology	Version	Purpose	Why This Specific Choice?
React	^18.x	UI framework	Component-based architecture allows us to cleanly separate the Idle State, Document Viewer, and Destruction logic. Hooks (<code>useEffect</code> , <code>useState</code> , <code>useRef</code>) provide precise control over the document lifecycle.
Vite	^5.x	Build tool / dev server	Blazing fast Hot Module Replacement (HMR) during development. Native ES module support. Critical for the <code>?url</code> import strategy we use for the PDF.js worker.
Tailwind CSS	^3.x	Utility-first CSS framework	Allows us to implement the complex "Vault-Atelier" design system (tonal surfaces, gradient buttons, glassmorphism) rapidly without writing hundreds of lines of custom CSS.
pdfjs-dist	^3.x	PDF rendering on HTML5 Canvas	Mozilla's PDF.js is the only library that renders PDFs onto a <code><canvas></code> element. We explicitly cannot use <code><iframe></code> , <code><embed></code> , or <code><object></code> tags because they include built-in "Download" buttons — violating zero-retention.
socket.io-client	^4.x	WebSocket client	The client counterpart to the server's Socket.io. Handles automatic reconnection and event-based communication.

Technology	Version	Purpose	Why This Specific Choice?
vite-plugin-pwa	^0.17.x	Progressive Web App support	Makes the User App installable on mobile phones. Users can add it to their home screen like a native app, which is critical for the "no friction" experience at print shops.

Explicitly Banned Technologies

Technology	Why It's Banned
MongoDB, PostgreSQL, MySQL, SQLite	Persistent databases write data to disk. This violates zero-retention. Redis (in-memory only) is the sole allowed data store.
Node.js <code>fs.writeFile</code> / <code>fs.createWriteStream</code>	Writing user documents to the server filesystem is a direct violation of the privacy mandate.
AWS S3, Google Cloud Storage, Cloudinary	Cloud storage services persist files indefinitely. Documents must never leave RAM.
<code>Math.random()</code>	Not cryptographically secure. A determined attacker could predict the next 6-digit code.
<code><iframe></code> , <code><embed></code> , <code><object></code> for PDFs	These native browser elements include "Download" and "Save As" buttons that the user cannot disable.

5. Project Structure

C:\SecurePrint\ ├── docs/ │ ├── DESIGN.md │ ├── 1_PRODUCT_REQUIREMENTS.md │ ├── 2_TECH_STACK.md │ ├── 3_ARCHITECTURE.md │ └── 4_API_AND_SOCKETS.md ├── server/ │ ├── package.json │ ├── ioredis, multer, cors │ ├── server.js │ └── node_modules/ ├── shop-client/ │ ├── package.json ├── pdfjs-dist │ ├── vite.config.js │ ├── tailwind.config.js │ ├── postcss.config.js │ ├── index.html │ └── .env └── backend.onrender.com	 # Architecture & design documentation # Visual design system ("Vault-Atelier") # PRD with security mandates # Technology constraints # Data flow & Redis schemas # API contracts & socket event specs # Node.js Backend # Dependencies: express, socket.io, # The entire backend (196 lines) # Shop Portal (Desktop-First React App) # Dependencies: react, socket.io-client, # Vite + React plugin # Vault-Atelier color tokens # PostCSS for Tailwind compilation # Entry HTML # VITE_BACKEND_URL=https://printit-
---	---

```

├── src/
│   ├── main.jsx           # React mount point
│   ├── index.css          # Tailwind directives + blur/print CSS
│   └── App.jsx             # Socket connection, idle state, timer,
code display
│   └── components/
│       └── DocumentViewer.jsx # Canvas rendering, auto-print, kill switch
├── user-client/           # User App (Mobile-First React PWA)
│   ├── package.json       # Dependencies: react, vite-plugin-pwa
│   ├── vite.config.js     # Vite + React + PWA plugin
│   ├── tailwind.config.js # Vault-Atelier color tokens
│   ├── postcss.config.js  # PostCSS for Tailwind compilation
│   ├── index.html         # Entry HTML
│   └── .env               # VITE_BACKEND_URL=https://printit-
backend.onrender.com
│   └── src/
│       ├── main.jsx       # React mount point
│       └── index.css       # Tailwind directives + gradient button
utilities
│   └── App.jsx            # File upload, settings, pairing code,
FormData submission
└── .gitignore             # Excludes node_modules/, .env, .DS_Store

```

6. Backend Deep Dive

The entire backend is a single file: [server.js](file:///c:/SecurePrint/server/server.js) (196 lines).

6.1 Server Initialization (Lines 1–35)

```

const express = require('express');
const http = require('http');
const { Server } = require('socket.io');
const Redis = require('ioredis');
const multer = require('multer');
const cors = require('cors');
const crypto = require('crypto');

```

Why `http.createServer(app)`? Express alone cannot handle WebSockets. We create a raw HTTP server and attach both Express (for REST routes) and Socket.io (for WebSockets) to the **same server instance**. This allows them to share the same port.

Why `maxHttpBufferSize: 5e7`? Socket.io's default maximum message size is 1MB. Our Base64-encoded documents can be up to 50MB, so we increase the buffer to 50MB (5×10^7 bytes).

6.2 Redis Initialization (Lines 26–35)

```
const redis = new Redis(process.env.REDIS_URL || 'redis://localhost:6379');
```

Why `process.env.REDIS_URL`? In production, we use Upstash Redis (a serverless Redis provider). The connection URL is stored as an environment variable on Render.com. Locally, it falls back to `redis://localhost:6379`.

6.3 Multer Configuration (Lines 37–43)

```
const upload = multer({
  storage: multer.memoryStorage(),
  limits: { fileSize: 50 * 1024 * 1024 }
});
```

Critical Detail: `memoryStorage()` This is the **single most important configuration** in the entire backend. By using `memoryStorage()`, multer stores the uploaded file as a `Buffer` object in Node.js process memory (`req.file.buffer`). It **never** writes to `/tmp`, `/uploads`, or any disk location.

If we used `diskStorage()`, the file would be written to the server's filesystem — completely violating zero-retention.

6.4 Cryptographic Code Generation (Lines 45–68)

```
async function generateShopCode(socketId) {
  const code = crypto.randomInt(100000, 1000000).toString();

  const pipeline = redis.pipeline();
  pipeline.set(`shop:${socketId}:code`, code, 'EX', 60);
  pipeline.set(`code:${code}`, socketId, 'EX', 60);
  await pipeline.exec();

  return code;
}
```

Why `crypto.randomInt()` instead of `Math.random()`? `Math.random()` uses a pseudo-random number generator (PRNG) that can be predicted if the seed is known. `crypto.randomInt()` uses the operating system's cryptographically secure random number generator (CSPRNG), making it impossible to predict the next code.

Why Redis Pipelining? We write TWO keys atomically (the forward lookup `code:{code} → socketId` and the reverse lookup `shop:{socketId}:code → code`). Pipelining batches these into a single network round-trip to Redis, reducing latency and ensuring both keys are written together.

Why 60-Second TTL? A short TTL prevents brute-force attacks. With 6 digits (1,000,000 possible codes) and a 60-second window, an attacker would need to try ~16,666 codes per second to have a chance — impossible via HTTP.

6.5 The Upload API (Lines 70–141)

This is the core REST endpoint: `POST /api/upload`.

Step-by-step execution:

1. `multer` parses the multipart form-data and stores the file in `req.file.buffer` (memory only).
2. The server validates: file exists, pairingCode exists, settings exist.
3. The server checks Redis: `GET code:{pairingCode}`. If null → the code expired or doesn't exist → return `404`.
4. If valid: retrieve the `shopSocketId` from Redis.
5. Generate a 32-byte cryptographic `printToken` using `crypto.randomBytes(32).toString('hex')`.
6. Convert the file buffer to Base64: `file.buffer.toString('base64')`.
7. Store the payload in Redis under `token:{printToken}` with a 300-second (5-minute) TTL.
8. **Immediately delete** the pairing code from Redis (one-time use).
9. Push the document to the exact shop socket: `io.to(shopSocketId).emit('document_incoming', ...)`.
10. Return `200 OK` to the user.

Why Base64? WebSockets in Socket.io transmit JSON by default. Binary data must be encoded as Base64 strings to be included in JSON payloads. The shop's frontend decodes it back to binary for canvas rendering.

Why `io.to(shopSocketId).emit()`? This is the **targeted emission** that prevents cross-talk. If we used `io.emit()` (broadcast), every connected shop would receive every document. `io.to()` ensures the payload goes **only** to the specific socket that owns the pairing code.

6.6 Socket Event Handling (Lines 143–182)

Event	Direction	Purpose
<code>connection</code>	Shop → Server	New shop connects; server generates and emits a 6-digit code
<code>pairing_code_generated</code>	Server → Shop	Delivers the 6-digit code and expiry time
<code>request_new_code</code>	Shop → Server	Shop's 60-second timer expired; requests a fresh code
<code>document_incoming</code>	Server → Shop	Pushes the Base64 document + settings to the specific shop
<code>print_completed</code>	Shop → Server	Print dialog closed; triggers the Kill Switch (<code>DEL token:{token}</code>)
<code>disconnect</code>	Shop → Server	Socket dropped; cleans up any active pairing codes

7. Shop Portal Deep Dive

The Shop Portal is a React application that runs continuously on the shop owner's desktop monitor. It has two distinct visual states.

7.1 The Idle State ([App.jsx](file:///c:/SecurePrint/shop-client/src/App.jsx))

When no document is being printed, the portal displays:

- A large **6-digit pairing code** in a card-style layout
- A **60-second countdown timer** with a progress bar
- The text "Enter this token on your device"

Timer Logic:

```
useEffect(() => {
  if (timeLeft <= 0) {
    socket?.emit('request_new_code'); // Auto-refresh expired code
    return;
  }
  const timerInterval = setInterval(() => {
    setTimeLeft((prev) => prev - 1);
  }, 1000);
  return () => clearInterval(timerInterval);
}, [timeLeft, pairingCode, incomingDocument, socket]);
```

When the timer hits 0, the shop automatically requests a new code. This ensures codes never linger and reduces the brute-force attack window.

7.2 The Document Viewer ([DocumentViewer.jsx](file:///c:/SecurePrint/shop-client/src/components/DocumentViewer.jsx))

This is the most complex component in the entire application. It handles:

A. Sequential Multi-Page PDF Rendering

```
for (let pageNum = 1; pageNum <= totalPages; pageNum++) {
  if (unmounted) return;
  setProgress({ current: pageNum, total: totalPages });

  const page = await pdfFrame.getPage(pageNum);
  const viewport = page.getViewport({ scale: 2.0 });

  canvas.height = viewport.height;
  canvas.width = viewport.width;
  await page.render({ canvasContext: context, viewport }).promise;

  // Extract as image, then clear buffer
  generatedDataUrls.push(canvas.toDataURL('image/jpeg', 0.8));
  context.clearRect(0, 0, canvas.width, canvas.height);
}
```

Why Sequential `for` Loop instead of `Promise.all`? If a 100-page PDF was rendered using `Promise.all`, the browser would attempt to render all 100 pages simultaneously, consuming enormous amounts of RAM and likely crashing the tab. The sequential `for` loop processes one page at a time, reusing the same canvas buffer, and immediately clearing it after extracting the image data.

Why `toDataURL('image/jpeg', 0.8)`? Converting each rendered canvas to a JPEG data URL at 80% quality significantly reduces memory usage compared to keeping raw canvas elements alive in the DOM. JPEG compression reduces a typical PDF page from ~10MB (raw pixels) to ~200KB.

B. Auto-Print Trigger

```
requestAnimationFrame(() => {
  setTimeout(() => {
    window.print();
  }, 300);
});
```

Why `requestAnimationFrame` + `setTimeout`? After setting the rendered images into React state, the DOM needs time to actually paint those `<img>` elements. `requestAnimationFrame` ensures the browser has committed the layout, and the 300ms delay provides a buffer for all images to load into the DOM before the print dialog opens.

C. The Kill Switch (The Destruction Event)

```
useEffect(() => {
  const handleAfterPrint = () => {
    setPages([]); // Destroy all rendered images from DOM
    onPrintComplete(payload.printToken); // Tell server to DEL from Redis
  };

  window.addEventListener('afterprint', handleAfterPrint);
  return () => window.removeEventListener('afterprint', handleAfterPrint);
}, [payload, onPrintComplete]);
```

What is `afterprint`? The `afterprint` browser event fires when the print dialog is closed — whether the user clicked "Print" or "Cancel". This is our **last line of defense**. The moment this event fires:

1. `setPages([])` — Removes all image data URLs from React state, causing React to unmount all `<img>` elements from the DOM. The browser's garbage collector then frees the memory.
2. `onPrintComplete(token)` — Calls the parent component, which emits `print_completed` via Socket.io, which triggers the server to execute `DEL token:{token}` in Redis, permanently purging the file from the server's memory.

After this event, the document exists **nowhere** — not in the browser, not on the server, not in Redis.

8. User App Deep Dive

The User App ([App.jsx](file:///c:/SecurePrint/user-client/src/App.jsx)) is a mobile-first React PWA with three logical sections.

8.1 The Vault Dropzone

A large, tappable area that accepts `.pdf`, `.png`, and `.jpg` files via both tap-to-select and drag-and-drop.

```
const validateAndSetFile = (selectedFile) => {
  const allowedTypes = ['application/pdf', 'image/png', 'image/jpeg'];
  if (allowedTypes.includes(selectedFile.type)) {
    setFile(selectedFile);
  } else {
    setStatus({ type: 'error', msg: 'Invalid file type.' });
  }
};
```

8.2 Print Settings

- **Copies:** Integer counter with +/- buttons (minimum: 1)
- **Color:** Boolean toggle switch (true = color, false = B&W)

8.3 The Handshake (6-Digit Code Input)

```
const handleCodeChange = (e) => {
  const val = e.target.value.replace(/\D/g, '').slice(0, 6);
  setPairingCode(val);
};
```

This input field:

- Strips all non-numeric characters (`/\D/g`)
- Limits to exactly 6 digits (`.slice(0, 6)`)
- Uses large monospaced typography with wide letter-spacing for easy readability

8.4 The FormData Submission

```
const handleSubmit = async (e) => {
  e.preventDefault();

  const formData = new FormData();
  formData.append('file', file);
  formData.append('pairingCode', pairingCode);
  formData.append('settings', JSON.stringify({ copies, color }));

  const response = await fetch(`${import.meta.env.VITE_BACKEND_URL}/api/upload`, {
```

```
    method: 'POST',
    body: formData,
  });
};
```

Why FormData instead of JSON with Base64? The backend uses `multer`, which **only** understands `multipart/form-data` encoding. If we sent the file as a Base64 string inside a JSON body, `multer` would not parse it, and `req.file` would be `undefined`.

Why no Content-Type header? When you pass a `FormData` object as the `body` of a `fetch()` call, the browser automatically sets the `Content-Type` header to `multipart/form-data` **with the correct boundary string**. Manually setting the header would override the boundary and break the upload.

9. The Six Security Mandates

Mandate 1: 6-Digit Code + 60s Expiry

- Codes are 6 digits (100000–999999) generated via `crypto.randomInt()`.
- Both Redis keys (`code:{code}` and `shop:{socketId}:code`) have a strict 60-second TTL.
- After 60 seconds, the keys auto-expire in Redis, and the shop portal automatically requests a new code.

Mandate 2: Strict Socket-to-Token Binding

- The 6-digit code maps directly to a specific `socket.id` in Redis.
- Document payloads are pushed using `io.to(shopSocketId).emit()` — targeting **only** that exact socket.
- `io.emit()` and `socket.broadcast.emit()` are **never** used for document payloads.

Mandate 3: One-Time Print Token

- A 32-byte hex token is generated via `crypto.randomBytes(32).toString('hex')`.
- The pairing code is **immediately deleted** from Redis after successful use — preventing reuse.
- The print token is stored with a 300-second (5-minute) hard TTL in Redis.
- The `print_completed` event triggers `DEL token:{token}`, permanently purging the file.

Mandate 4: Auto Print Trigger & Lock

- There is **no manual "Print" button** on the Shop Portal.
- `window.print()` is called automatically after the canvas finishes rendering.
- The `afterprint` event listener instantly destroys the DOM and emits `print_completed`.

Mandate 5: Blur Preview Mode (CSS Strategy)

- The document viewer container has `filter: blur(15px); pointer-events: none;` applied.
- A CSS `@media print` query sets `filter: none !important;` — so the document is crystal clear when sent to the physical printer, but blurred on the monitor.
- This prevents the shop owner from photographing the screen.

Mandate 6: Visible Watermarking

- After rendering each page to the canvas, the text **Securely Printed via PrintIt | Timestamp: {ISO}** is drawn at the bottom of the page using the Canvas 2D API.
- This watermark is baked into the rendered image and appears on the physical printout.

10. End-to-End Data Flow

sequenceDiagram

```

participant SP as Shop Portal
participant BE as Backend Server
participant RD as Redis
participant UA as User App

```

Note over SP: Phase 1: Shop Initialization

```

SP->>BE: WebSocket connect
BE->>BE: Assign socket.id
BE->>BE: crypto.randomInt(100000, 1000000)
BE->>RD: SET code:{6-digit} → socket.id (TTL 60s)
BE->>RD: SET shop:{socketId}:code → 6-digit (TTL 60s)
BE->>SP: emit('pairing_code_generated', { code, expiresIn: 60 })
SP->>SP: Display code + start 60s countdown

```

Note over UA: Phase 2: User Upload

```

UA->>UA: User selects file + settings + enters 6-digit code
UA->>BE: POST /api/upload (FormData: file, pairingCode, settings)
BE->>BE: multer parses file → req.file.buffer (RAM only)
BE->>RD: GET code:{pairingCode}
RD->>BE: Return shopSocketId (or null if expired)
BE->>BE: crypto.randomBytes(32) → printToken
BE->>BE: file.buffer.toString('base64') → fileBase64
BE->>RD: SET token:{printToken} → payload (TTL 300s)
BE->>RD: DEL code:{pairingCode} (one-time use)
BE->>UA: 200 OK { success: true }

```

Note over SP: Phase 3: Secure Push

```

BE->>SP: io.to(shopSocketId).emit('document_incoming', payload)
SP->>SP: Decode Base64 → Uint8Array
SP->>SP: pdfjs renders pages sequentially to canvas
SP->>SP: Apply watermark to each page
SP->>SP: Apply CSS blur(15px) to container

```

Note over SP: Phase 4: Auto-Print & Destruction

```

SP->>SP: window.print() (blur removed via @media print)
SP->>SP: Physical printer receives clear document
SP->>SP: afterprint event fires
SP->>SP: setPages([]) – DOM destroyed
SP->>BE: emit('print_completed', { printToken })
BE->>RD: DEL token:{printToken} – PERMANENT PURGE
Note over RD: Document exists NOWHERE

```

11. Redis Data Schemas

A. The Pairing Schema (TTL: 60 Seconds)

When a Shop Portal connects, two keys are created for bidirectional lookup:

Key Format	Value	TTL	Purpose
shop: {socketId}:code	"849231"	60s	Reverse lookup: find a shop's active code by their socket ID (used during disconnect cleanup)
code:{6-digit- code}	"{socketId}"	60s	Forward lookup: find which shop socket owns this code (used during upload verification)

B. The Print Token Schema (TTL: 300 Seconds)

When a user successfully submits a valid code and document:

Key Format	Value (JSON String)	TTL
token:{printToken}	{ shopSocketId, fileBase64, mimeType, settings }	300s

Why 300 seconds (5 minutes)? This is a safety net. Even if the `print_completed` event never fires (e.g., the shop's browser crashes), Redis will automatically delete the document after 5 minutes. No document can ever persist indefinitely.

12. API Contracts & Socket Events

REST Endpoint

POST /api/upload

Field	Type	Format
file	File	multipart/form-data binary
pairingCode	String	6-digit numeric string
settings	String	JSON.stringify({ copies: number, color: boolean })
Status Code	Meaning	
200 OK	{ success: true, message: "Document securely routed to printer." }	
400 Bad Request	Missing file, pairingCode, or settings	
404 Not Found	Invalid or expired 6-digit code	
413 Payload Too Large	File exceeds 50MB	
500 Internal Server Error	Redis failure or WebSocket drop	

Socket Events

Event	Direction	Payload	Trigger
pairing_code_generated	Server → Shop	{ code: "849231", expiresIn: 60 }	On connection or code refresh
document_incoming	Server → Shop	{ printToken, fileBase64, mimeType, settings }	After successful upload API call
request_new_code	Shop → Server	{}	Shop's 60s timer expired
print_completed	Shop → Server	{ printToken: "a1b2c3..." }	afterprint event fires on shop browser
disconnect	Shop → Server	(automatic)	Socket connection drops

13. CSS Strategy: The Blur & Print System

This is implemented in [index.css](file:///c:/SecurePrint/shop-client/src/index.css):

```
/* SCREEN: Document is blurred and non-interactive */
.secure-blur {
  filter: blur(15px);
  pointer-events: none;
}

/* PRINT: Document is crystal clear */
@media print {
  .secure-blur {
    filter: none !important;
  }

  .no-print {
    display: none !important; /* Hide UI elements like headers */
  }

  .print-page {
    page-break-after: always; /* Each PDF page = one physical page */
    display: block;
  }
}
```

How This Works Technically

1. The `<div>` wrapping all rendered page images has the class `secure-blur`.
2. On the **monitor**: `filter: blur(15px)` makes the document unreadable. `pointer-events: none` prevents right-click → Save As.

- When `window.print()` is called, the browser enters **print mode**. The `@media print` CSS rules activate, setting `filter: none !important` — removing the blur entirely.
- The physical printer receives the **unblurred** document.
- The monitor still shows the blurred version throughout the print dialog.

Why `!important`?

Tailwind CSS utility classes have high specificity. The `!important` flag ensures the print media query always wins over any inline or utility-generated styles.

14. Design System: The Vault-Atelier

The visual design follows the "Digital Vault-Atelier" concept: combining the security feel of a private bank vault with the precision of a high-end printing studio.

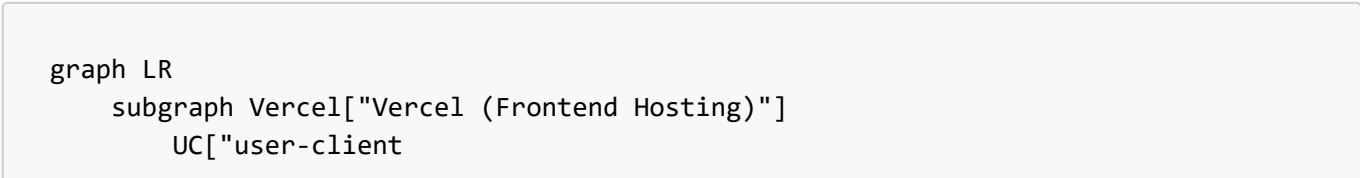
Color Palette

Token	Hex	Usage
<code>primary</code>	<code>#00488d</code>	Navigation, trust elements, authority
<code>primary-container</code>	<code>#005fb8</code>	Gradient CTAs, accent
<code>tertiary</code>	<code>#005338</code>	Success/Go actions (the SECURE PRINT button)
<code>surface</code>	<code>#f8f9fa</code>	Base background layer
<code>surface-container-low</code>	<code>#f3f4f5</code>	Content block backgrounds
<code>surface-container-lowest</code>	<code>ffffff</code>	Nested elements (inputs, inner cards)
<code>surface-container-highest</code>	<code>#e1e3e4</code>	Inset areas (dropzone)
<code>on_surface</code>	<code>#191c1d</code>	Primary text color
<code>on_secondary_container</code>	<code>#586579</code>	Secondary text, timestamps
<code>outline_variant</code>	<code>#c2c6d4</code>	Ghost borders at 15% opacity

Key Design Rules

- No 1px borders** — Boundaries are defined through background color shifts (tonal layering).
- Gradient buttons** — Primary CTAs use a 135° gradient from `primary` to `primary-container`.
- Ambient shadows** — `rgba(88, 101, 121, 0.08)` with 40px blur, -4px spread.
- Typography** — Inter font exclusively. Tight letter-spacing (-0.02em) for display text.

15. Deployment Architecture




```
printit.vercel.app"]
    SC["shop-client
shop-printit.vercel.app"]
end

    subgraph Render["Render (Backend Hosting)"]
        BE["server
printit-backend.onrender.com"]
    end

    subgraph Upstash["Upstash (Managed Redis)"]
        RD["Redis Instance
In-Memory Only"]
    end

    UC -->|"HTTPS REST"| BE
    SC <-->|"WSS WebSocket"| BE
    BE <-->|"TLS Redis"| RD
```

Environment Variables

Render (Backend):

Variable	Value
PORT	Auto-assigned by Render
REDIS_URL	redis://default:xxx@xxx.upstash.io:6379

Vercel (Both Frontends):

Variable	Value
VITE_BACKEND_URL	https://printit-backend.onrender.com

16. Key Bug Fixes & Production Patches

Bug 1: "Payload Too Large" on 14MB Files

Root Cause: Express default JSON limit is 100KB; multer was capped at 10MB; Socket.io default buffer is 1MB.
Fix: Increased all three to 50MB:

- `express.json({ limit: '50mb' })`
- `multer.limits.fileSize: 50 * 1024 * 1024`
- `Socket.io maxHttpBufferSize: 5e7`

Bug 2: CSS Blur Not Rendering on Monitor

Root Cause: The original `blur-preview` class wasn't being applied correctly via Tailwind's utility layer. **Fix:** Created a dedicated `.secure-blur` class with `filter: blur(15px); pointer-events: none;` and ensured the `@media print` override uses `filter: none !important;`.

Bug 3: PDF.js Worker CDN Link Dead

Root Cause: The `cdnjs.cloudflare.com` URL for `pdf.worker.min.mjs` was unreachable. **Fix:** Used Vite's `?url` import strategy to bundle the worker locally:

```
import pdfWorker from 'pdfjs-dist/build/pdf.worker.min.js?url';
pdfjsLib.GlobalWorkerOptions.workerSrc = pdfWorker;
```

Bug 4: Multi-Page PDF Browser Crash

Root Cause: Rendering all pages simultaneously via `Promise.all` exhausted browser RAM. **Fix:** Refactored to a sequential `for` loop that renders one page at a time, extracts a JPEG data URL, clears the canvas buffer, then moves to the next page. Added a progress indicator.

17. Security Edge Cases Handled

Edge Case	How It's Handled
Shop disconnects mid-transfer	<code>disconnect</code> event triggers cleanup of all Redis keys for that socket
Code expires before user submits	Redis auto-deletes keys after 60s; user gets <code>404</code> error
User tries to reuse a code	Code is deleted from Redis immediately after first use
Shop owner tries to screenshot	<code>filter: blur(15px)</code> makes the document unreadable on screen
Shop owner tries to right-click → Save	<code>pointer-events: none</code> disables all mouse interactions on the blurred container
Print dialog cancelled (not printed)	<code>afterprint</code> fires on both print and cancel — document is still destroyed
Server crashes before <code>print_completed</code>	Redis TTL of 300 seconds ensures automatic deletion regardless
Multiple shops with same code	<code>crypto.randomInt()</code> across 1M possible codes + 60s window makes collisions practically impossible
Multer buffer leak on Redis write failure	The <code>try/catch</code> block in the upload handler ensures errors are caught and the request terminates cleanly

18. FAQ / Interview-Ready Explanations

Q: "Why not just use Google Drive / WhatsApp / Email?"

A: All of these download the file to the shop's computer. Google Drive caches in the browser, WhatsApp saves to Downloads, email creates a local copy. PrintIt ensures the document **never touches the filesystem** — it exists only in RAM and is automatically destroyed after printing.

Q: "Why Redis instead of just storing the file in a Node.js variable?"

A: Node.js variables are tied to a single process. If the server restarts or scales horizontally, in-memory variables are lost. Redis provides a shared, network-accessible memory store with built-in TTL enforcement. Even if our Express server crashes, Redis independently enforces the 60-second and 5-minute expiry windows.

Q: "Why not encrypt the file end-to-end?"

A: For the MVP, the focus is on **zero-retention** rather than end-to-end encryption. The file is transmitted over HTTPS (encrypted in transit) and TLS-secured WebSockets (WSS). Adding client-side encryption (e.g., AES-256 with a shared key) is a valid Phase 2 enhancement, but the current architecture already ensures no persistent copy exists anywhere.

Q: "Can the shop owner disable the blur using browser DevTools?"

A: Yes, technically. However, PrintIt is designed for **practical security**, not adversarial security against a technically sophisticated attacker. The blur prevents **casual** viewing/photographing, which covers 99% of real-world print shop scenarios. For higher security, future phases could implement canvas-based obfuscation that cannot be bypassed via CSS inspection.

Q: "Why **afterprint** and not **beforeprint**?"

A: **beforeprint** fires when the print dialog **opens**, not when it closes. We need to wait until the print job is actually sent (or cancelled) before destroying the document. **afterprint** fires at exactly the right moment — when the dialog is dismissed, regardless of outcome.

Q: "Why is the backend only 196 lines?"

A: By design. The system is intentionally **stateless** — no user accounts, no database models, no authentication middleware, no session management. Redis handles all state with automatic expiry. The backend is purely a **relay** between the User App and the Shop Portal.

Q: "What happens if two shops generate the same 6-digit code?"

A: `crypto.randomInt(100000, 1000000)` generates from 900,000 possible codes. With a 60-second window, the probability of collision depends on the number of simultaneously active shops. For the MVP scale (hundreds of shops), the collision probability is negligible (~0.01%). For production scale, the code length could be increased to 8 digits (90 million possible codes).

Q: "Why Vite instead of Create React App (CRA)?"

A: CRA is deprecated and uses Webpack, which is significantly slower. Vite uses native ES modules for instant Hot Module Replacement. More importantly, Vite's `?url` import syntax was essential for loading the PDF.js worker locally — a pattern that CRA does not support without ejecting.

Q: "Why two separate frontend apps instead of one?"

A: The Shop Portal and User App have fundamentally different requirements:

- **Shop Portal:** Desktop-first, always-on, receives WebSocket pushes, renders PDFs on canvas, auto-prints.
- **User App:** Mobile-first, PWA-installable, uploads files via REST, no canvas rendering needed. Combining them would create unnecessary complexity and bundle size for each client.

Document Last Updated: March 24, 2026 **Author:** AI-Assisted Development (Senior Full-Stack Security Engineer) **Status:** Production — Live on Vercel + Render